HUET Séverin
GIORDANI Enzo
PETROVIC Hugo

06/03/2026

Projet SAÉ - Refonte de l'Application de Placement

Table des matières

Table des matières 1
1. Bilan du travail de la semaine (9 au 13 Mars) 2
1.1 Introduction et contexte sur le travail de la BDD 2
1.2 Audit de normalisation (1FN à 4FN) 2
A) Violation de la 1FN — plan.donnee (valeur non atomique) 2
B) Violation de la 2FN — groupe.nb_etud (dépendance partielle) 2
C) Question 3FN — salle.id_dpt (dépendance transitive potentielle) 3
D) Risque de 4FN — devoir_groupe et devoir_promo (dépendances multivaluées) 3
1.3. Conformité avec le cahier des charges 4
A) Entités manquantes — Corrections apportées 4
1.4. Optimisations techniques 5
A) Contraintes d'intégrité référentielle — table salle 5
B) Contrainte d'unicité sur le placement 5
C) Index supplémentaires 6
D) Optimisation du type de données — promotion.annee 6
E) Blocage des dates invalides — devoir.date_devoir 6
1.5. Sécurité et qualité des données 7
A) Remplacement du hachage MD5 par bcrypt 7
B) Nettoyage des données de test corrompues 7
1.6. Récapitulatif 9
2. Planning prévisionnel pour la semaine suivante (16 au 13 Mars) 10

1. Bilan du travail de la semaine (9 au 13 Mars)

1.1 Introduction et contexte sur le travail de la BDD

Cette semaine nous avons retravaillé la base de données que nous avons fais la semaine dernière. Nous avons travaillé sur les 4 axes.

- Analyse de la normalisation (1FN à 4FN)
- Vérification de la conformité avec le cahier des charges
- Optimisation technique (types, index, contraintes d'intégrité)
- Sécurité des données et cohérence métier

1.2 Audit de normalisation (1FN à 4FN)

A) Violation de la 1FN — plan.donnee (valeur non atomique)

Problème identifié

L'attribut donnée de la table plan stocke une grille de salle entière sous forme d'une chaîne de caractères encodée (ex. : '333333333333333-111111111111111-...'). La Première Forme Normale exige que chaque attribut contienne une valeur atomique, c'est-à-dire indivisible.

Impact

Impossible d'interroger ou de modifier une cellule individuelle de la grille en SQL pur.
Toute opération sur le plan nécessite un parsing côté applicatif, ce qui couplé à un format non documenté, crée une dette technique significative.

Décision

Refactorisation hors scope de la SAÉ 4.01. Documentée comme dette technique avec la structure cible suivante :

```
CREATE TABLE plan_cellule (
  id_plan   INT UNSIGNED NOT NULL,
  x         TINYINT UNSIGNED NOT NULL,
  y         TINYINT UNSIGNED NOT NULL,
  type_cell ENUM('LIBRE','OCCUPE','COULOIR','MUR') NOT NULL DEFAULT 'LIBRE',
  PRIMARY KEY (id_plan, x, y),
  FOREIGN KEY (id_plan) REFERENCES plan(id_plan) ON DELETE CASCADE
);
```

En attendant la refactorisation : le format d'encodage de donnee doit être documenté dans le README et le parsing centralisé dans une seule fonction utilitaire.

B) Violation de la 2FN — groupe.nb_etud (dépendance partielle)

Problème identifié

La colonne nb_etud dans la table groupe est une valeur calculable à tout moment par la requête COUNT(*) FROM etudiant WHERE id_groupe = X. La Deuxième Forme Normale interdit les attributs pouvant être déterminés à partir d'un sous-ensemble de la clé ou d'autres données existantes.

Impact

Risque permanent de désynchronisation : si un étudiant est ajouté ou supprimé sans mise à jour de nb_etud, la donnée devient incohérente.
Redondance inutile de données.

Correction appliquée

Suppression de la colonne et création d'une vue dédiée :

```
ALTER TABLE groupe DROP COLUMN nb_etud;


CREATE OR REPLACE VIEW v_groupe_effectif AS
SELECT g.id_groupe, g.nom_groupe, g.id_promo,
       COUNT(e.id_etudiant) AS nb_etud
FROM groupe g
LEFT JOIN etudiant e ON e.id_groupe = g.id_groupe
GROUP BY g.id_groupe, g.nom_groupe, g.id_promo;
```

Le LEFT JOIN garantit qu'un groupe sans aucun étudiant apparaît dans la vue avec nb_etud = 0, au lieu de disparaître du résultat.

C) Question 3FN — salle.id_dpt (dépendance transitive potentielle)

Problème analysé

Si un bâtiment appartient toujours à un seul département, la chaîne id_salle → id_bat → id_dpt constituerait une dépendance transitive, violant la Troisième Forme Normale.

Décision après consultation

Un même bâtiment physique peut contenir des salles appartenant à des départements différents (ex. : salles INFO et salles SD dans le même bâtiment). La dépendance fonctionnelle est donc directe : id_salle → id_dpt, indépendamment de id_bat. Le schéma est conforme à la 3FN sur ce point. Aucune modification requise.

D) Risque de 4FN — devoir_groupe et devoir_promo (dépendances multivaluées)

Problème identifié

Un même devoir pouvait apparaître simultanément dans devoir_groupe (lié à des groupes spécifiques) et dans devoir_promo (lié à une promotion entière). Sans contrainte, ces deux associations pouvaient exprimer des faits contradictoires : un devoir affecté à une promo entière mais aussi à seulement certains de ses groupes.

Règle métier validée

Les deux tables sont mutuellement exclusives. Un devoir est soit destiné à des groupes précis (ex. devoir de TP), soit à une promotion entière (ex. examen final). L'affectation à une promotion inclut implicitement tous ses groupes.

Correction appliquée — Triggers de cohérence

```
DELIMITER $$
CREATE TRIGGER trg_check_devoir_groupe_insert
BEFORE INSERT ON devoir_groupe FOR EACH ROW
BEGIN
  IF EXISTS (SELECT 1 FROM devoir_promo WHERE id_devoir = NEW.id_devoir) THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Conflit : ce devoir est affecté à une promotion
entière.';
  END IF;
END$$

CREATE TRIGGER trg_check_devoir_promo_insert
BEFORE INSERT ON devoir_promo FOR EACH ROW
BEGIN
  IF EXISTS (SELECT 1 FROM devoir_groupe WHERE id_devoir = NEW.id_devoir) THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Conflit : ce devoir est affecté à des groupes
spécifiques.';
  END IF;
END$$
DELIMITER ;
```

1.3. Conformité avec le cahier des charges

A) Entités manquantes — Corrections apportées

placement au premier rang

Le cahier des charges mentionne explicitement que certains étudiants doivent être placés au premier rang. Les colonnes tiers_temps et mob_reduite existaient, mais aucun champ ne couvrait ce cas.

Correction appliquée

```
ALTER TABLE etudiant
  ADD COLUMN premier_rang TINYINT(1) NOT NULL DEFAULT 0
  COMMENT 'Contrat pédagogique : placement obligatoire au premier rang';
```

Type de salle

Le cahier des charges exige qu'au moins un TD et un amphi soient gérés. La table salle ne disposait d'aucun champ formel pour distinguer les types. La colonne intercal ne suffit pas à exprimer cette sémantique.

Correction appliquée

```
ALTER TABLE salle
  ADD COLUMN type_salle ENUM('TD','TP','AMPHI','AUTRE') NOT NULL DEFAULT 'TD'
  COMMENT 'Type de salle pour le placement';
```

1.4. Optimisations techniques

A) Contraintes d'intégrité référentielle — table salle

Problème identifié

Les clés étrangères de la table salle vers batiment, departement et plan n'avaient pas de comportement ON DELETE défini. En MariaDB, l'absence de ON DELETE équivaut à RESTRICT implicite, mais cette règle n'étant pas déclarée explicitement, elle n'était pas documentée ni garantie selon les versions.

Correction appliquée

Les FK existantes ont été supprimées puis recréées avec les comportements explicites. Remarque importante : les noms réels des contraintes dans le schéma sont fk_v2_salle_bat, fk_v2_salle_dpt et fk_v2_salle_plan (et non fk_salle_* comme indiqué initialement — erreur corrigée après vérification dans le dump SQL).

```
ALTER TABLE salle
  DROP FOREIGN KEY fk_v2_salle_bat,
  DROP FOREIGN KEY fk_v2_salle_dpt,
  DROP FOREIGN KEY fk_v2_salle_plan;


ALTER TABLE salle
  ADD CONSTRAINT fk_v2_salle_bat
    FOREIGN KEY (id_bat) REFERENCES batiment(id_bat)
    ON DELETE RESTRICT ON UPDATE CASCADE,
  ADD CONSTRAINT fk_v2_salle_dpt
    FOREIGN KEY (id_dpt) REFERENCES departement(id_dpt)
    ON DELETE RESTRICT ON UPDATE CASCADE,
  ADD CONSTRAINT fk_v2_salle_plan
    FOREIGN KEY (id_plan) REFERENCES plan(id_plan)
    ON DELETE RESTRICT ON UPDATE CASCADE;
```

ON DELETE RESTRICT : empêche la suppression d'un bâtiment, département ou plan tant que des salles y sont rattachées. ON UPDATE CASCADE : propage automatiquement les modifications d'identifiants.

B) Contrainte d'unicité sur le placement

Problème identifié

La table placement ne comportait aucune contrainte garantissant qu'une place physique (salle + devoir + coordonnées x/y) ne peut être occupée que par un seul étudiant. Deux étudiants auraient pu théoriquement se voir attribuer la même place.

Correction appliquée

```
ALTER TABLE placement
  ADD UNIQUE KEY uq_placement_place (id_devoir, id_salle, place_x, place_y);
```

C) Index supplémentaires

Justification générale

Les index accélèrent les requêtes de filtrage et de jointure. Quatre index manquants ont été identifiés sur des colonnes régulièrement interrogées par la logique applicative.

Index créé	Table.Colonne	Justification
idx_etudiant_tiers_temps	etudiant.tiers_temps	Filtrage rapide des étudiants à temps majoré
idx_etudiant_mob_reduite	etudiant.mob_reduite	Filtrage rapide des étudiants à mobilité réduite
idx_devoir_heure	devoir.heure_devoir	Détection de conflits horaires entre devoirs
idx_matiere_nom	matiere.nom_mat	Recherche de matières par intitulé

```
CREATE INDEX idx_etudiant_tiers_temps ON etudiant (tiers_temps);
CREATE INDEX idx_etudiant_mob_reduite  ON etudiant (mob_reduite);
CREATE INDEX idx_devoir_heure          ON devoir    (heure_devoir);
CREATE INDEX idx_matiere_nom           ON matiere   (nom_mat);
```

D) Optimisation du type de données — promotion.annee

Problème identifié

La colonne annee était déclarée en INT(11), soit 4 octets par ligne. Une année universitaire (ex. 2025) tient sur un SMALLINT UNSIGNED (2 octets, plage 0–65535). Économie de 2 octets par ligne, sans impact sur la logique.

Correction appliquée

```
ALTER TABLE promotion
  MODIFY COLUMN annee SMALLINT UNSIGNED DEFAULT NULL
  COMMENT 'Année universitaire (ex: 2025)';
```

E) Blocage des dates invalides — devoir.date_devoir

Problème identifié

De nombreuses lignes dans devoir contenaient date_devoir = '0000-00-00', valeur invalide résultant d'insertions de test sans données. En MariaDB 10.3, la clause CHECK est acceptée syntaxiquement mais non appliquée. Un trigger BEFORE INSERT est nécessaire pour un contrôle effectif.

Correction appliquée

```
DELIMITER $$
CREATE TRIGGER trg_devoir_date_check
BEFORE INSERT ON devoir FOR EACH ROW
BEGIN
  IF NEW.date_devoir = '0000-00-00' THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'date_devoir invalide : 0000-00-00 interdit';
  END IF;
END$$
DELIMITER ;
```

1.5. Sécurité et qualité des données

A) Remplacement du hachage MD5 par bcrypt

Problème identifié

Les mots de passe enseignants étaient stockés avec l'algorithme MD5, cassé cryptographiquement depuis 2004. Un hash MD5 peut être inversé par force brute ou via des tables arc-en-ciel en quelques secondes.

Décision

Le type VARCHAR(255) de la colonne pass est déjà compatible avec bcrypt (60 caractères). Aucune modification de schéma n'est nécessaire. Le changement est purement applicatif (PHP).

Migration des données existantes

```
-- Invalider les hashes MD5 existants (longueur fixe de 32 caractères)
-- pour forcer un reset au premier login
UPDATE enseignant SET pass = NULL WHERE LENGTH(pass) = 32;
```

Côté PHP — remplacement obligatoire

```
// Supprimer partout
$hash = md5($password);


// Création / modification du mot de passe
$hash = password_hash($password, PASSWORD_BCRYPT);


// Vérification au login
if (password_verify($password, $hash_from_db)) { /* authentifié */ }
```

password_hash() et password_verify() sont disponibles nativement depuis PHP 5.5. Aucune dépendance supplémentaire n'est requise.

B) Nettoyage des données de test corrompues

Problème identifié

La table devoir contenait de nombreuses lignes de test non nettoyées : date_devoir = '0000-00-00', nom_devoir = '', duree_devoir = '00:00:00'. Ces données invalides pouvaient fausser le comportement de l'application et les statistiques.

Procédure appliquée

Identification préalable avant suppression :

```
SELECT id_devoir, nom_devoir, date_devoir, duree_devoir FROM devoir
WHERE date_devoir = '0000-00-00' OR nom_devoir = '' OR duree_devoir = '00:00:00';
```

Suppression (les cascades ON DELETE CASCADE propagent aux tables liées : devoir_groupe, devoir_promo, devoir_salle, placement, surveille) :

```
DELETE FROM devoir
WHERE date_devoir = '0000-00-00'
   OR nom_devoir = ''
   OR duree_devoir = '00:00:00';
```

1.6. Récapitulatif

#	Objet	Type d'opération	Justification
1	groupe.nb_etud	**SUPPRESSION**	Violation 2FN — valeur calculable, risque de désynchronisation
2	v_groupe_effectif	**CRÉATION (vue)**	Remplacement de nb_etud par un COUNT() dynamique garanti exact
3	FK salle → batiment/dpt/plan	**MODIFICATION**	Ajout explicite ON DELETE RESTRICT ON UPDATE CASCADE
4	placement — unicité de place	**AJOUT (contrainte)**	Empêche deux étudiants d'occuper la même place physique
5	etudiant.premier_rang	**AJOUT (colonne)**	Exigence cahier des charges — contrat pédagogique
6	salle.type_salle	**AJOUT (colonne)**	Exigence cahier des charges — distinction TD / Amphi / TP
7	promotion.annee	**MODIFICATION (type)**	INT → SMALLINT UNSIGNED — optimisation mémoire
8	trg_devoir_date_check	**CRÉATION (trigger)**	Blocage des dates 0000-00-00 (CHECK non appliqué en MariaDB 10.3)
9	trg_check_devoir_groupe/promo	**CRÉATION (triggers)**	Exclusivité mutuelle devoir_groupe / devoir_promo — 4FN
10	4 index manquants	**CRÉATION (index)**	tiers_temps, mob_reduite, heure_devoir, nom_mat — performance
11	enseignant.pass MD5 → bcrypt	**MODIFICATION (PHP)**	MD5 cassé depuis 2004 — sécurité critique
12	Données corrompues devoir	**SUPPRESSION (données)**	Nettoyage lignes de test invalides avant livraison
13	plan.donnee	DOCUMENTÉ (dette)	Violation 1FN — refactorisation hors scope, structure cible documentée
14	salle.id_dpt	VALIDÉ (conforme)	Pas de violation 3FN — dépendance directe confirmée après consultation

2. Planning prévisionnel pour la semaine suivante (16 au 13 Mars)

Les objectifs de la semaine 16-20 Mars reprennent le planning initialement prévu pour cette semaine, qui n'a pas pu être entamé en raison du temps consacré à l'audit et la stabilisation de la BDD.

POUR LA BDD : Séverin

3.1 Nettoyer le fichier connexion.php. Pour PDO (Estimation : ~ 2h)

3.2 Modifier l'inscription/insertion des profs. (hashage BCrypt) (Estimation : ~ 1h)

POUR L'architecture MVC : Enzo (potentiellement aidé par Séverin)

2.1 Déplacer la logique d'authentification de login.php vers un contrôleur dédié (AuthController). (Estimation : ~ 2h)

2.2 Isoler les requêtes SQL éparpillées dans les anciens fichiers (gest_salle.php, etc.) pour les centraliser dans des classes Modèles dédiées. (Estimation : ~ 3h à 4h — tâche de fond à lisser sur la semaine)

POUR UI/UX : Hugo (potentiellement aidé par Séverin)

1.1 Supprimer les balises <iframe> dans crea_salle.php et transformer la création de salle en formulaire classique géré par étapes en JavaScript. (Estimation : ~ 3h)

1.2 Remplacer les pop-ups JavaScript bloquantes (alert()) par des messages d'erreur et de validation intégrés en HTML/CSS dans les vues. (Estimation : ~ 2h)